

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview	1
2. Background & Business Objective	1
3. Product Purpose	2
4. System Functionalities	2
5. User Stories & Use Cases	3
6. Features Included (Scope Definition)	3
7. Features Not Included (Scope Control)	3
8. Assumptions & Constraints	4
9. Risks & Questions Throughout Development	4

Project Name: Jalan-Ready

Version: 2.0

1. Project Overview

- **Problem Statement:** In rapidly developing regions like Selangor, road maintenance is managed through fragmented federal (JKR Federal), state (JKR State), and local municipal authorities (MBPJ, MBSA, etc.). Traditional reporting relies on manual citizen complaints or scheduled patrols, resulting in:
 - (1) Jurisdictional confusion where citizens report to wrong departments
 - (2) Static maintenance treating repairs as one-off tasks without considering environmental data or historical recurrence
 - (3) Lack of stateful tracking with no automated follow-up or escalation logic.
- **Target Domain:** Infrastructure governance and automated road defect reporting for Selangor State, covering Federal roads, State roads, and Municipal/City roads across 9 JKR districts and 11 Local Councils (PBT).
- **Proposed Solution Summary:** SIRCa is an Autonomous Infrastructure Governance Engine that uses a General Language Model (GLM) as its central reasoning core. It transforms unstructured citizen reports (text + images) into stateful, adaptive work orders, intelligently assigns jurisdiction across Selangor authorities, and dynamically optimizes contractor routing while considering weather, traffic, and historical recurrence patterns.

2. Background & Business Objective

- **Background of the Problem:** Infrastructure management currently suffers from three critical failures:

- (1) Fragmented & unstructured data—legacy systems cannot intelligently categorize diverse inputs like potholes, alligator cracking, standing water, or faded lines, nor assess urgency based on proximity to hospitals or schools
 - (2) Rigid, scripted workflows that fail when faced with real-world ambiguity—if a GPS coordinate is missing or an API call fails, the process stops rather than reasoning an alternative path
 - (3) Non-adaptive execution that does not account for dynamic variables like weather—painting road lines during heavy Selangor rain wastes resources.
- **Importance of Solving This Issue:** Accelerating road hazard response from days to hours can prevent accidents, reduce vehicle damage, and save lives. Breaking the jurisdictional confusion barrier enables citizens to report issues without needing to know which authority manages which road. The system ensures no safety hazard "falls through the cracks" by autonomously routing to the correct JKR district or local council across all 9 Selangor districts.
 - **Strategic Fit / Impact:** Aligns with Malaysia's Smart City and Digital Government initiatives. Demonstrates AI-native governance where an agentic workflow replaces rigid scripts. The Z.AI GLM integration enables reasoning-based routing, priority calculation, weather-aware scheduling, and recurrence detection—capabilities impossible with traditional rule-based systems.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-enabled autonomous governance partner for road infrastructure maintenance in Selangor, automating the end-to-end workflow from citizen report submission to jurisdiction dispatch with intelligent reasoning at every stage.

3.2. Intended Users (Target Audience):

- Citizens (Public)
- JKR District Officers
- Local Council Officers (PBT)
- Contractors
- System Administrators

4. System Functionalities

4.1. Description: The system operates as a multi-agent LangGraph workflow with Z.AI GLM as the central reasoning orchestrator. It accepts unstructured text descriptions and images, performs computer vision analysis (YOLOv8), reasons about jurisdiction and priority, checks weather history and forecasts, detects recurrence patterns, and generates structured work orders dispatched to the correct Selangor authority.

4.2. Key Functionalities:

- **Conversational Reporting:** Chat-style interface where users describe road issues naturally and upload photos. GLM asks targeted follow-up questions if information is missing.
- **Multi-Modal Input Understanding:** GLM combines text descriptions with YOLOv8 vision analysis (pothole, crack, faded line detection with confidence scores)..]
- **Jurisdiction Auto-Routing (Selangor-Specific):** System determines if road is Federal/State (routes to JKR District) or Municipal/City (routes to specific Local Council) based on reverse geocoding.
- **Dynamic Priority Calculation:** GLM calculates urgency score (1-100) based on defect severity, road class, traffic volume, proximity to critical infrastructure (schools, hospitals), and recurrence flag.
- **Weather-Aware Scheduling (Enhanced):** GLM checks past weather (last 72 hours) for sub-base saturation flags and forecast weather (48 hours) to avoid scheduling repairs during rain.

- **Recurring Problem Investigation:** GLM queries database for similar reports within 100m radius in last 90 days. If ≥ 2 previous reports exist, escalates priority and adds root-cause investigation note.
- **Stateful Workflow Memory:** LangGraph checkpoints save exact workflow state. Users can resume mid-report. Reports track states: NEW $\rightarrow$ AWAITING_INFO $\rightarrow$ REPORTED $\rightarrow$ IN_PROGRESS $\rightarrow$ ESCALATED $\rightarrow$ RESOLVED.
- **External Email Dispatch:** System sends structured work orders to correct JKR District or Local Council email addresses with complete assessment data.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Do state the LLM used (Z AI GLM) along with justification why and what capabilities made it the right choice for the product's core functionalities.

4.3.2. Prompting Strategy: Describe your team's approach [e.g. zero-shot, few shot or multi-step agentic prompting]. So, explain the reason why the strategy chosen does fit your use case.

4.3.3. Context & Input Handling: Describe how the system that your team developed handles the unstructured or inputs that are oversized. So, what's the maximum input size your product does accept before it actually does truncation or chunking or rejection? So, what's going to happen to those inputs that exceed the limit?

4.3.4. Fallback & Failure Behavior: What does the developed product do when the model returns off-topic or unusable and most importantly hallucinated response? So, is there any mechanism for retrying, a human escalation path or even a graceful error state?

5. User Stories & Use Cases

- **User Stories:**

1. As a citizen, I want to report a road defect by simply typing what I see and uploading a photo, so that I don't need to know which government department handles that road.
2. As a JKR officer, I want to receive automatically routed work orders with priority scores and weather context, so that I can dispatch repair teams efficiently.
3. As a contractor, I want optimized daily routes that avoid traffic congestion, so that I can complete more repairs in less time.
4. As a system administrator, I want the GLM to handle missing data gracefully by asking follow-up questions, so that incomplete reports don't get lost.
5. As a citizen, I want to track my report status (Submitted → Scheduled → In Progress → Completed), so that I know when the issue will be fixed.

- **Use Cases (Main Interactions):**

Prompt to UI Flow: User is logged into Streamlit app

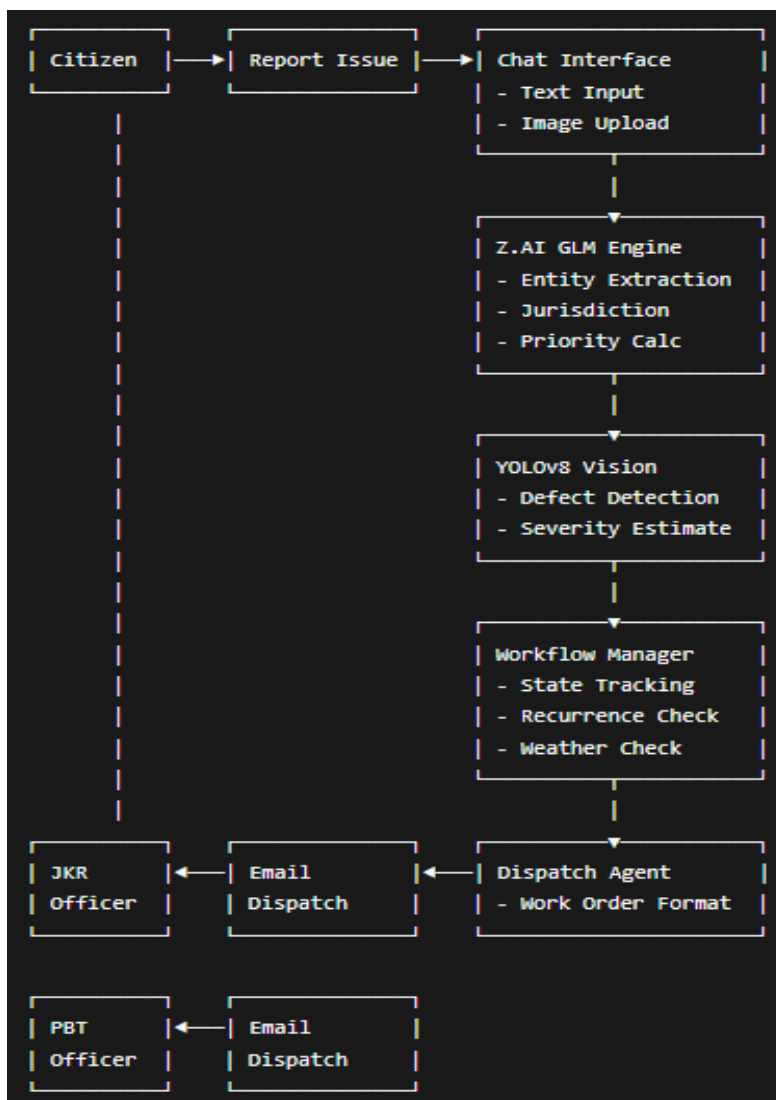
Flow Generation:

1. User types "Large crack on Jalan SS2/72, PJ" + uploads photo. 2. YOLOv8 analyzes photo → "Longitudinal crack, moderate width". 3. Geocoding identifies Municipal road under MBPJ jurisdiction. 4. GLM checks database for recurrence (finds 1 previous report). 5. GLM calculates priority: Medium. 6. GLM checks past weather: Heavy rain yesterday → adds "Sub-base wet" flag. 7. GLM schedules repair for tomorrow 10 AM. 8. System sends work order email to MBPJ. 9. User sees confirmation with priority and estimated response. Report state = REPORTED, tracked in SQLite database

Prompt to UI Flow: User submits report without location

Flow Generation: 1. User types "There's a pothole" + uploads photo. 2. GLM detects missing GPS coordinates. 3. System moves report to AWAITING_INFO state. 4. GLM generates conversational prompt: "We see the pothole, but where is it located? Can you name a nearby landmark or road?" 5. Citizen replies with location. 6. Workflow resumes with complete data. Report transitions from AWAITING_INFO → REPORTED

- **Use Case Diagram:**



6. Features Included (Scope Definition)

- Conversational chat-based reporting
- YOLOv8 vision analysis
- GLM-powered entity extraction
- Jurisdiction auto-routing
- Dynamic priority calculation
- Weather-aware scheduling
- Recurrence detection
- Stateful SQLite tracking
- Email dispatch to authorities
- User dashboard
- Streamlit web interface
- LangGraph workflow orchestration

7. Features Not Included (Scope Control)

- SMS/WhatsApp integration
- Real-time contractor mobile app
- Payment processing Not applicable to road reporting
- Multi-language support (beyond Malay/English)
- Live traffic API integration
- Officer dashboard with write-back
- Production-scale concurrency
- Mobile native apps

8. Assumptions & Constraints

- **LLM Cost Constraint:** Estimated 500-1,000 tokens per user session. Design decision: Truncate long conversation history to last 5 exchanges before sending to GLM. Cache jurisdiction mapping queries for 24 hours.
- **Technical Constraints:** YOLOv8 runs locally in ONNX format (no GPU required). Email dispatch uses SMTP with environment variables for credentials. No production database—SQLite file-based.

- **Performance Constraints:** YOLOv8 inference: 200-500ms per image on CPU. GLM API latency: 2-4 seconds. Total workflow: ~5-8 seconds per report. Generation limit: 5 concurrent users max.
- **User Input:** System requires human prompts to initiate. Cannot generate reports without user input. Missing data triggers AWAITING_INFO state rather than automatic rejection.
- **Geographic Constraint:** System only covers Selangor State. Federal roads outside Selangor, State roads in other states, and roads in other Malaysian states are out of scope.
- **Data Availability:** Assumes Open-Meteo API availability for weather data. Assumes Google Maps Geocoding API availability for reverse geocoding.

9. Risks & Questions Throughout Development

- **Jurisdiction Ambiguity:** GLM flags ambiguous cases for manual review; system includes override capability in officer dashboard
- **API Rate Limiting:** Implement caching of geocoding results; use batch requests where possible; add exponential backoff on rate limit errors
- **GLM Hallucination:** Structured JSON output schema enforced; validation layer checks that extracted district exists in known list; fallback to default (JKR Petaling) if hallucination detected
- **Recurrence False Positive:** 100m radius is configurable; officers can override recurrence flag via dashboard
- **Email Delivery Failure:** Implement retry queue with exponential backoff (retry at 5min, 30min, 2hr, then flag for manual review)
- **Weather API Unavailability:** Fallback to "unknown weather" status; scheduling proceeds without weather delay; system logs incident for review